Model-Driven Test Design (MDTD)

Definition

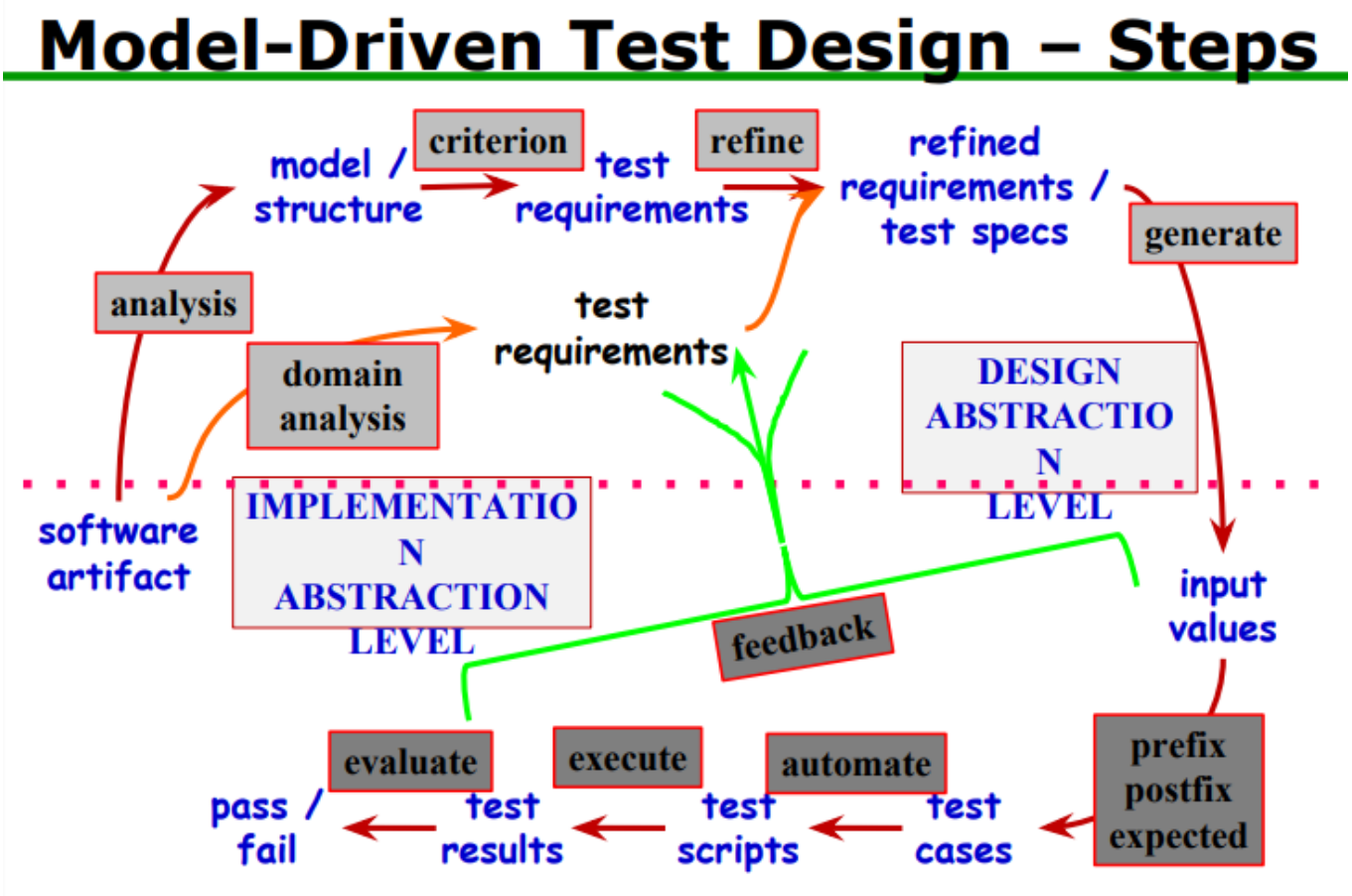
Model-Driven Test Design (MDTD) is a systematic software testing approach in which **models of a software system are used to derive test requirements, test specifications, test cases, test scripts, and test data**.

In MDTD, the behavior or structure of the software is represented by a model, and this model is systematically used to design and execute effective test cases.

Basic Process

Software Artifact → Model/Structure → Test Requirements → Refined Test Requirements/Test Specifications → Test Cases → Test Scripts → Test Execution → Test Evaluation

Figure:



Steps of Model-Driven Test Design

1. Analyze the Software Artifact

First, the software artifact, such as source code, requirements, or system behavior, is analyzed to understand its functionality and structure.

2. Build a Model

A suitable model is created to represent the important behavior, structure, or control flow of the software system.

3. Derive Test Requirements

The model is analyzed to identify the **test requirements** that must be satisfied during testing.

4. Refine Test Requirements

The identified requirements are refined into more detailed and precise **test specifications**.

5. Design Test Cases

Based on the test specifications, appropriate **test cases, test paths, and input values** are designed.

6. Automate the Tests

Where applicable, the designed test cases are converted into **automated test scripts**.

7. Execute the Tests

The test scripts are executed using the selected input values, and the actual test results are collected.

8. Evaluate the Results

The actual results are compared with the expected results to determine whether each test **passes or fails**.

9. Provide Feedback

The test results and failures provide feedback that can be used to improve the model, test requirements, test specifications, or test cases.

Example of MDTD

Consider the following Java method:

```
public int indexOf(Node n)
{
    for (int i=0; i<path.size(); i++)
    {
        if (path.get(i).equals(n))
            return i;
    }
    return -1;
}
```

This method searches for a node `n` in a path.

- If the node is found, its **index is returned**.
- If the node is not found, **−1 is returned**.

A **Control Flow Graph (CFG)** can be created from this program to represent its different execution paths.

Figure:

Draw the Control Flow Graph of the given program here.

The CFG can then be used to derive test requirements based on a selected coverage criterion, such as **Edge-Pair Coverage**.

Edge-Pair Coverage

Edge-Pair Coverage is a structural testing criterion in which every possible **pair of consecutive edges** in a control-flow graph should be covered by at least one test path.

For example, possible test paths may include:

- `[1, 2, 5]`
- `[1, 2, 3, 2, 5]`
- `[1, 2, 3, 2, 3, 4]`

These paths are selected to satisfy the required edge-pair coverage of the control-flow graph.

Figure:

Draw the Control Flow Graph and mark the required test paths here.

Advantages of MDTD

1. **Systematic Testing:** Provides a structured and systematic way to design tests.
2. **Better Test Coverage:** Helps identify important execution paths and coverage requirements.
3. **Early Test Design:** Test requirements can be derived early from the model.
4. **Automation Support:** Test cases and scripts can be automated.
5. **Repeatability:** The same model can be used to generate repeatable tests.
6. **Reduced Human Error:** Systematic model-based test generation reduces the chance of missing important test conditions.
7. **Easy Maintenance:** When the system model is updated, the corresponding test design can also be refined.

Conclusion

Model-Driven Test Design provides a systematic bridge between software models and executable tests. By deriving test requirements, test specifications, test cases, and test scripts from models, MDTD improves test coverage, consistency, automation, and overall testing effectiveness.

⋆ Exam-এ মনে রাখার shortcut

Model → Requirements → Specifications → Test Cases → Test Scripts → Execute → Evaluate

এই flow-টা ঠিকভাবে লিখে, **CFG/MDTD figure সুন্দরভাবে এঁকে**, তারপর steps ও advantages দিলে উত্তরটা যথেষ্ট **complete এবং full-marks oriented** হবে।

Coverage Criteria

Coverage Criteria is a set of rules that specifies **which parts of a software system must be tested**. It helps testers select an effective set of test cases that provides maximum testing coverage while using the minimum number of tests.

Easy to Remember:
Coverage Criteria = Rules that tell us what to cover during testing.

Why Do We Need Coverage Criteria?

Modern software can have an enormous number of possible input combinations.

Practical Example

```
double computeAverage(int A, int B, int C)
```

Suppose each variable is a **32-bit integer**.

- A has about **4 billion** possible values.
- B has about **4 billion** possible values.
- C has about **4 billion** possible values.

Total possible test cases:

4 Billion × 4 Billion × 4 Billion = More than 80 Octillion test cases

It is impossible to execute all of them.

Therefore, testers use **Coverage Criteria** to select the most important test cases.

Two Major Problems in Software Testing

1. How do we search?

Problem:
There are too many possible inputs.

Solution:
Coverage Criteria helps testers choose the most useful test cases instead of testing every possible input.

Practical Example

Instead of testing all possible ages,

Test only:

- Age = -1
- Age = 0
- Age = 18
- Age = 60

- Age = 100

These values cover important situations while using very few test cases.

2. When do we stop testing?

Problem:
How do we know enough testing has been performed?

Solution:
Testing can stop when the selected Coverage Criterion has been satisfied.

Practical Example

Suppose a program contains **20 statements**.

When every statement has been executed at least once,

Statement Coverage = 100%

The tester knows that the coverage goal has been achieved.

Goals of Coverage Criteria

Coverage Criteria helps achieve two important goals.

1. Thorough Search

It ensures that important parts of the software are tested.

Example

An ATM system should test:

- Correct PIN
- Incorrect PIN
- Empty PIN
- Maximum PIN attempts

Instead of testing only one situation.

2. Avoid Overlapping Tests

It reduces unnecessary duplicate test cases.

Example

Testing:

- Login with wrong password
- Login with wrong password again

These two tests are almost identical.

Instead, replace the second one with

- Login using an empty password

This increases coverage.

Test Criterion

A **Test Criterion** is a collection of rules and procedures that defines **what must be tested** and **how testing requirements are determined**.

Easy Memory Tip:
Test Criterion = Testing Rules

Examples

- Cover every statement.
- Cover every branch.
- Cover every functional requirement.

Test Requirement

A **Test Requirement** is a specific software element that **must be tested** to satisfy a particular Test Criterion.

Easy Memory Tip:
Test Requirement = The actual item that must be covered.

Examples

If the Test Criterion is

"Cover every statement."

Then

Every individual statement becomes a **Test Requirement**.

If the Test Criterion is

"Cover every functional requirement."

Then

Every software function becomes a **Test Requirement**.

Practical Example

Consider the following program:

```
if(age >= 18)
    System.out.println("Eligible");
else
    System.out.println("Not Eligible");
```

Test Criterion

Cover every statement.

Test Requirements

Test Case 1

```
Age = 20
```

Output

```
Eligible
```

This covers the first statement.

Test Case 2

```
Age = 15
```

Output

```
Not Eligible
```

This covers the second statement.

Now every statement has been tested.

Difference Between Test Criterion and Test Requirement

Test Criterion	Test Requirement
Defines the testing rule	Defines the specific item to test
General concept	Specific target
Example: Statement Coverage	Example: Execute Statement 1
Example: Branch Coverage	Example: Execute True Branch

What is White-box Testing?

White-box Testing (also called **Structural Testing** or **Glass-box Testing**) is a software testing technique in which the tester **knows the internal source code, program logic, and structure** of the software and creates test cases based on the code.

Easy Definition (1 line):

White-box Testing tests the internal code and logic of a software.

Software Example

Suppose we have this program:

```
if (marks >= 40)
    System.out.println("Pass");
else
    System.out.println("Fail");
```

Another Best Example:

```
if(pinCorrect)
{
    if(balance>=amount)
        Withdraw();
}
```

A White-box Tester looks inside the code.

What is Black-box Testing?

Black-box Testing (also called **Functional Testing**) is a software testing technique in which the tester **does not know the internal source code** and tests only the software's inputs and outputs.

Easy Definition (1 line):

Black-box Testing checks whether the software works correctly without looking at the source code.
It does **NOT** check the code

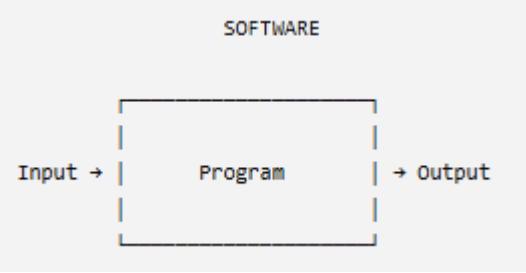
Software Example

Suppose the Login System:

Tester enters invalid Username and Password
Username = admin and Password = wrong123
Output
Invalid Password

Tester enters valid Username and Password
Username = admin and Password = admin123
Output
Login Successful

The tester never looks at the source code.
They only verify the output.
This is Black-box Testing.



White-box Testing	Black-box Testing
Also called Structural Testing	Also called Functional Testing
Tests internal code	Tests external functionality
Source code is visible	Source code is hidden
Programming knowledge required	Programming knowledge not required
Focuses on code, branches, statements	Focuses on inputs and outputs
Usually performed by Developers	Usually performed by QA/Testers
Checks how the software works	Checks what the software does
Measures code coverage	Measures requirement coverage

Types of Test Activities

Test Activities are the different tasks performed during the software testing process to ensure that the software is tested efficiently and produces high-quality results.

There are **four main Test Activities**:

1. Test Design
2. Test Automation
3. Test Execution
4. Test Evaluation

Memory Trick: D-A-E-E

This is the complete lifecycle of software testing.

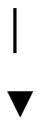
Requirements

1. Test Design

2. Test Automation

3. Test Execution

4. Test Evaluation



Final Test Report

1. Test Design

Test Design is the process of creating effective test cases that can detect defects in the software.

It is considered **the most technical and most important activity** in software testing.

Easy Definition

Test Design means planning what to test and how to test it.

Why is it Important?

A well-designed test case can find many bugs.

A poorly designed test case may miss important defects.

Therefore,

Good Test Design = Better Software Quality

There are **two approaches** to Test Design:

- Criteria-Based Test Design
- Human-Based Test Design

A. Criteria-Based Test Design

In **Criteria-Based Test Design**, test cases are created using predefined testing criteria such as Statement Coverage, Branch Coverage, Path Coverage, etc.

The tester follows systematic rules instead of guessing.

Practical Example

Suppose the program is

```
if(age >=18)
    Print("Adult");
else
    Print("Minor");
```

The test designer creates test cases to ensure

- True branch is executed.
- False branch is executed.

Example Test Cases

Input	Output
20	Adult
15	Minor

Both branches are covered.

This is Criteria-Based Test Design.

B. Human-Based Test Design

Human-Based Test Design depends on the tester's **experience, domain knowledge, and creativity** rather than only following testing criteria.

It focuses on situations that automated rules may miss.

Why is it Needed?

Sometimes Coverage Criteria cannot identify unusual or real-world situations.

Human thinking helps discover these hidden problems.

Practical Example

Suppose you are testing an **Online Shopping Website**.

Coverage Criteria checks

✓ Add Product

✓ Remove Product

✓ Payment

A Human Tester also thinks

"What happens if the user clicks the **Pay** button ten times?"

"What happens if the internet disconnects during payment?"

"What happens if the user refreshes the page?"

These situations are found through human thinking.

Criteria-Based	Human-Based
Uses testing rules	Uses experience and creativity
Requires programming knowledge	Requires domain knowledge
Systematic	Practical
Code-oriented	User-oriented

2. Test Automation

Test Automation is the process of using software tools or scripts to execute test cases automatically instead of manually.

Easy Definition

Test Automation means using programs to perform testing automatically.

Practical Example

Suppose a Login Page must be tested every day.

Instead of typing

Username

Password

Login

again and again,

an automation script performs the test automatically.

This saves hours of work.

Important Challenge

The lecture mentions two important concepts:

Observability

Can we easily observe the software's output?

Example

Can the automation tool verify whether **"Login Successful"** is displayed?

Controllability

Can the automation tool control the software?

Example

Can it automatically enter Username and Password?

Why is Automation Needed?

Automation

- Saves time
- Reduces manual effort
- Executes repeated tests quickly

3. Test Execution

Test Execution is the process of running the designed test cases on the software.

Easy Definition

Test Execution means **running the test cases and recording the results.**

Practical Example

Suppose the Test Case is

Username = admin

Password = admin123

The Test Executor simply

- Opens the software
- Enters the data
- Clicks Login
- Records the result

No coding is required.

4. Test Evaluation

Test Evaluation is the process of analyzing the test results and deciding whether the software behaves correctly.

Easy Definition

Test Evaluation means **checking whether the software passed or failed.**

Practical Example

Expected Output

```
Total = 5000
```

Actual Output

```
Total = 4500
```

The evaluator investigates

- Is this a software bug?
- Is the expected result wrong?
- Is the business rule incorrect?

Another Example

Suppose a Banking System calculates interest.

Only a banking expert knows whether the calculation is correct.

Therefore,

Domain knowledge is essential.

Key Points

✓ Compares Expected vs Actual Output

✓ Requires domain knowledge

✓ Determines Pass or Fail

✓ Final quality decision is made here

Summary Table (Very Important for Exam)

Activity	Main Purpose	Required Skills	Example
Test Design	Create test cases	Programming, Testing, CS	Design login test cases
Test Automation	Automate testing	Programming	Selenium script for login
Test Execution	Run test cases	Basic computer skills	Execute login test
Test Evaluation	Analyze results	Domain knowledge, Testing	Compare expected vs actual result

Other Activities / Other Test Activities

Besides the four main activities, there are three supporting activities.

1. Test Management

Definition: Plans and manages the entire testing process.

Responsibilities

- Sets testing policies
- Organizes the testing team
- Communicates with developers
- Chooses testing criteria
- Decides how much automation is needed

Example:
A Test Manager decides that the project will use **80% automated testing** and **20% manual testing**, assigns team members, and monitors progress.

2. Test Maintenance

Definition: Updates and reuses test cases as the software evolves.

Responsibilities

- Reuse existing test cases
- Remove outdated tests
- Update test scripts after software changes
- Store tests in configuration control

Example:
A new **"Forgot Password"** feature is added to the system. Existing login test cases are updated to include this new functionality.

3. Test Documentation

Definition: Records all testing information so that every test can be understood and traced.

Responsibilities

- Document why each test exists
- Link tests to requirements (Traceability)
- Store documentation with automated tests

Example:
For a login test case, documentation includes:

- Requirement ID: **REQ-101**
- Purpose: Verify valid user login
- Expected Result: Login successful

Organizing the Test Team

A mature testing organization assigns the **right people to the right tasks**.

Typical Team Structure

- **1 Test Designer** works with several:
    - Test Automators
    - Test Executors
    - Test Evaluators

Important Points

- Increased automation reduces the need for manual Test Executors.
- Assigning the wrong person to the wrong task decreases efficiency and job satisfaction.
- Test Evaluators should be free to add extra test cases based on their **domain knowledge**, because engineering rules alone may miss real-world scenarios.
- **The four test activities are different**, so they should not always be handled by the same people.

Exam Note: Although many organizations use the same people for all four activities, this is **not considered a best practice**, because each activity requires different skills and expertise.

What is a Dummy Node?

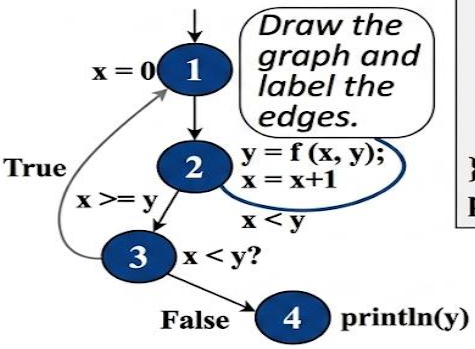
A Dummy Node is a non-executable node in a Control Flow Graph (CFG). It does not represent any program statement; it is used only to connect or merge different control flow paths.

বাংলা:

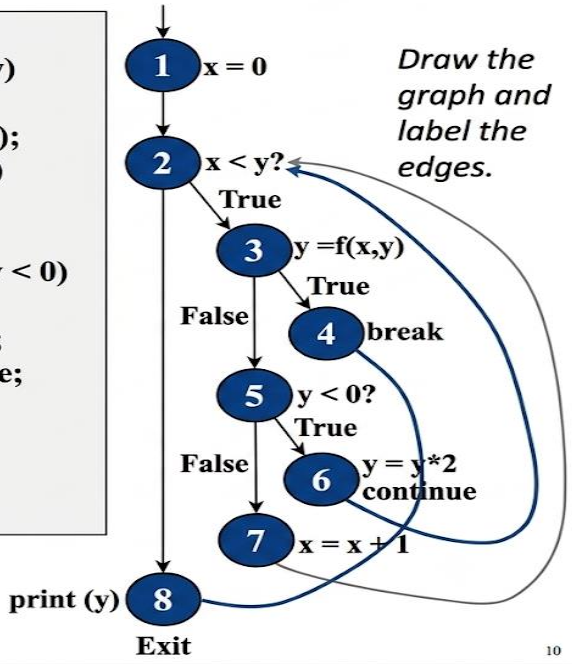
Dummy Node হলো Control Flow Graph (CFG)-এর একটি non-executable (খালি) node, যা কোনো program statement নির্দেশ করে না। এটি শুধুমাত্র বিভিন্ন control flow path-কে সংযুক্ত (connect) বা একত্রিত (merge) করার জন্য ব্যবহৃত হয়।

CFG : do Loop, break and continue

```
x = 0;
do
{
   y = f (x, y);
   x = x + 1;
} while (x < y);
println (y)
```



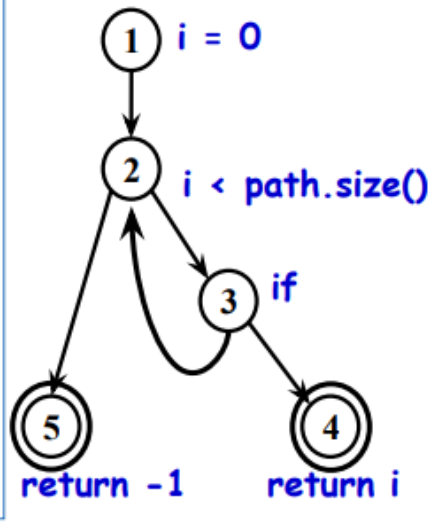

```
x = 0;
while (x < y)
{
   y = f (x, y);
   if (y == 0)
   {
      break;
   } else if (y < 0)
   {
      y = y*2;
      continue;
   }
   x = x + 1;
}
print (y);
```



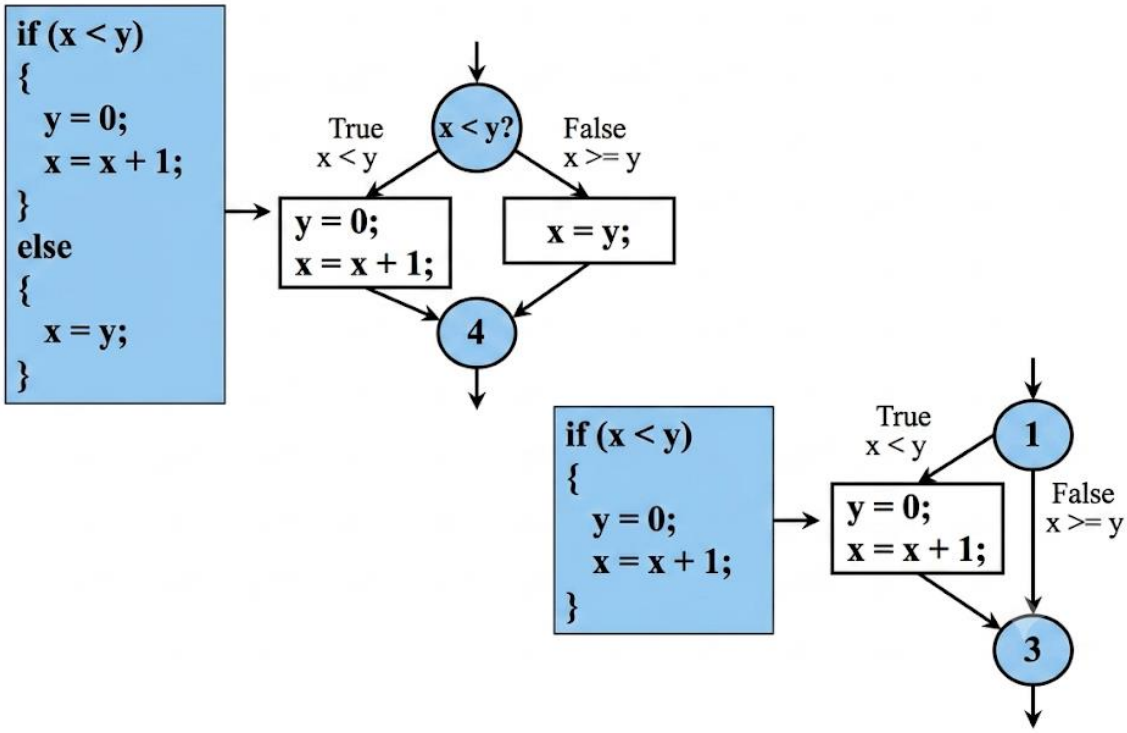
Software Artifact : Java Method

```
/**
   * Return index of node n at the
   * first position it appears,
   * -1 if it is not present
*/
public int indexOf (Node n)
{
   for (int i=0; i < path.size(); i++)
      if (path.get(i).equals(n))
         return i;
   return -1;
}
```

Control Flow Graph



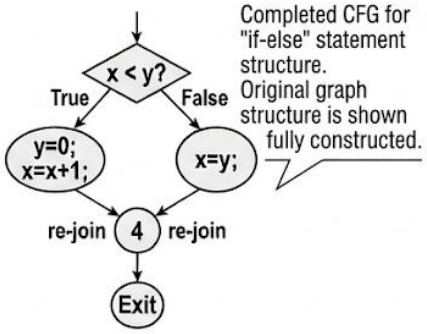
CFG : The if Statement



Mastering Control Flow Graphs (CFGs): A Comprehensive Guide

Section A: if-else Statement

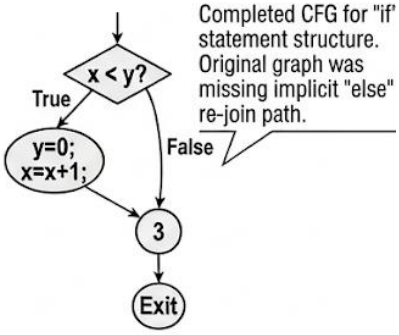
```
if (x < y)
{
  y = 0;
  x = x + 1;
}
else
{
  x = y;
}
```



Completed CFG for "if-else" statement structure. Original graph structure is shown fully constructed.

Section B: if Statement

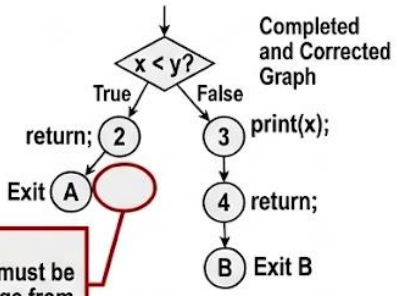
```
if (x < y)
{
  y = 0;
  x = x + 1;
}
```



Completed CFG for "if" statement structure. Original graph was missing implicit "else" re-join path.

Section C: if-Return Statement

```
if (x < y)
{
  return;
}
print (x);
return;
```



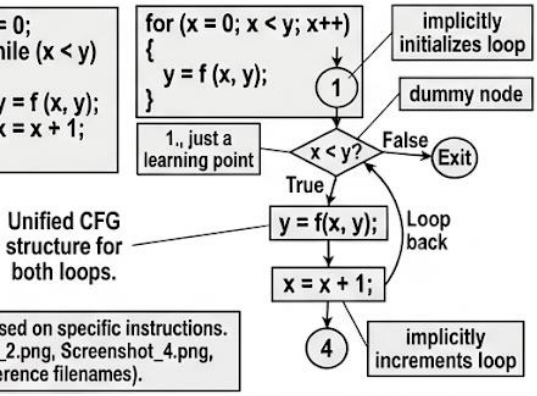
Completed and Corrected Graph

CRUCIAL RULE:
The return nodes must be and there is no edge from node 2 to 3.

Section D: while & for Loops

```
x = 0;
while (x < y)
{
  y = f (x, y);
  x = x + 1;
}
```

```
for (x = 0; x < y; x++)
{
  y = f (x, y);
}
```



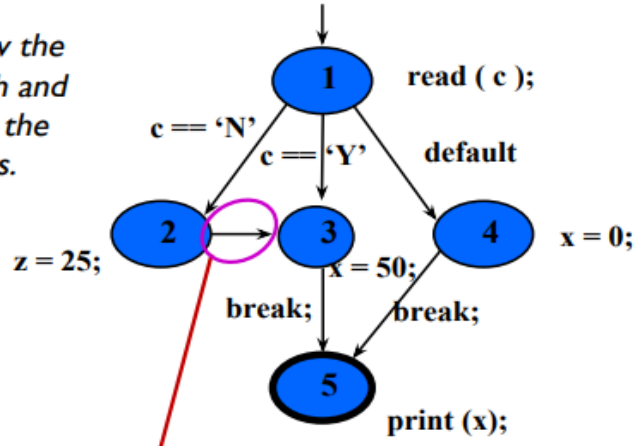
Unified CFG structure for both loops.

All graphs are completed and corrected based on specific instructions. Lessons derived from data in Screenshot_2.png, Screenshot_4.png, Screenshot_5.png (verbatim reference filenames).

CFG : The case (switch) Structure

```
read ( c) ;
switch ( c )
{
  case 'N':
    z = 25;
  case 'Y':
    x = 50;
    break;
  default:
    x = 0;
    break;
}
print (x);
```

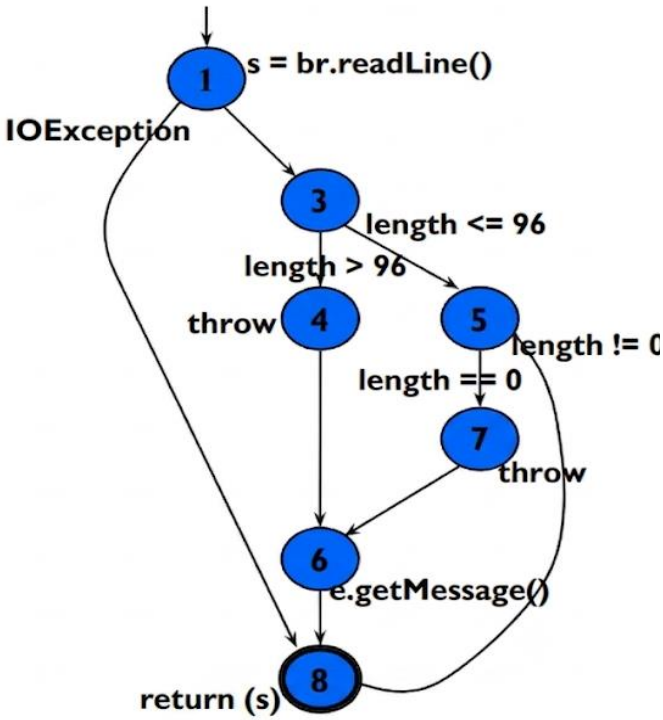
Draw the graph and label the edges.



Cases without breaks fall through to the next case

CFG : Exceptions (try-catch)

```
try
{
  s = br.readLine();
  if (s.length() > 96)
    throw new Exception
      ("too long");
  if (s.length() == 0)
    throw new Exception
      ("too short");
} (catch IOException e) {
  e.printStackTrace();
} (catch Exception e) {
  e.getMessage();
}
return (s);
```



Full CFG path analysis of try-catch flow.